Ex: No:5	Implementation of Predictive Parser
Date:	

AIM:

To implement Predictive parser using C Programming

ALGORITHM:

Step 1: Start the program.
Step 2: Read grammar productions from the user until "quit" is entered.
Step 3: Store each production by separating LHS and RHS, and split RHS using '/' for multiple productions.
Step 4: Identify and store all terminals and non-terminals, and add '$' as end marker.
Step 5: Initialize FIRST and FOLLOW sets for all non-terminals.
Step 6: Compute FIRST set by checking the first symbol of each production and recursively adding values.
Step 7: Initialize FOLLOW of the start symbol with '$'.
Step 8: Compute FOLLOW set using rules for terminals, non-terminals, and end positions.
Step 9: Construct the parsing table using FIRST and FOLLOW sets.
Step 10: Display the parsing table with non-terminals and terminals.
Step 11: Read the input string and append '$', then initialize stack with '$' and start symbol.
Step 12: Parse the string using stack operations and parsing table; accept if both reach '$', otherwise reject.
Step 13: Stop the program

PROGRAM:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
struct tran {
   char node;
   int n, k, g, fi, fo;
   char t[10][20];
   char first[20], follow[20];
   char par[20][20];
} b[20];
int count = 0, c = 0;
char a[10][20], t[20], stack[50], string[50], e;
/* -------- utility -------- */
void add_unique(char *arr, int *len, char val) {
   for (int i = 0; i < *len; i++)
      if (arr[i] == val) return;
   arr[(*len)++] = val;
}
/* -------- FIRST -------- */
void first(int x) {
   b[x].k = 1;
   for (int i = 0; i <= b[x].n; i++) {
      char sym = b[x].t[i][0];
```

```
        if (!isupper(sym)) {
            add_unique(b[x].first, &b[x].fi, sym);
        } else {
            for (int j = 0; j < count; j++) {
                if (b[j].node == sym) {
                    if (b[j].k == 0)
                        first(j);
                    for (int k = 0; k < b[j].fi; k++) {
                        if (b[j].first[k] != '@')
                            add_unique(b[x].first, &b[x].fi, b[j].first[k]);
                    }
                }
            }
        }
    }
}
/* -------- FOLLOW -------- */
void follow(int x) {
    b[x].g = 1;
    if (x == 0)
        add_unique(b[x].follow, &b[x].fo, '$');
    for (int i = 0; i < count; i++) {
        for (int j = 0; j <= b[i].n; j++) {
            for (int k = 0; k < strlen(b[i].t[j]); k++) {
                if (b[i].t[j][k] == b[x].node) {
                    if (b[i].t[j][k + 1] != '\0') {
                        char next = b[i].t[j][k + 1];
                        if (!isupper(next)) {
                            add_unique(b[x].follow, &b[x].fo, next);
                        } else {
                            for (int l = 0; l < count; l++) {
                                if (b[l].node == next) {

                                    for (int m = 0; m < b[l].fi; m++) {
                                        if (b[l].first[m] != '@')
                                            add_unique(b[x].follow, &b[x].fo, b[l].first[m]);
                                    }
                                    // if epsilon present
                                    for (int m = 0; m < b[l].fi; m++) {
                                        if (b[l].first[m] == '@') {
                                            if (b[i].g == 0)
                                                follow(i);
                                            for (int n = 0; n < b[i].fo; n++)
                                                add_unique(b[x].follow, &b[x].fo, b[i].follow[n]);
                                        }
                                    }
```

```
                    }
                }
            }

        } else {
            if (b[i].g == 0)
                follow(i);
            for (int l = 0; l < b[i].fo; l++)
                add_unique(b[x].follow, &b[x].fo, b[i].follow[l]);
        }
    }
  }
 }
}
/* -------- PARSE TABLE -------- */
void get_parse_table() {
    for (int i = 0; i < count; i++) {
        for (int j = 0; j <= b[i].n; j++) {
            char first_sym = b[i].t[j][0];
            // epsilon production
            if (first_sym == '@') {
                for (int k = 0; k < b[i].fo; k++) {
                    for (int n = 0; n < c; n++) {
                        if (t[n] == b[i].follow[k])
                            strcpy(b[i].par[n], "@");
                    }
                }
            }
            // terminal
            else if (!isupper(first_sym)) {
                for (int n = 0; n < c; n++) {
                    if (t[n] == first_sym)
                        strcpy(b[i].par[n], b[i].t[j]);
                }
            }
            // non-terminal
            else {
                for (int l = 0; l < count; l++) {
                    if (b[l].node == first_sym) {
                        for (int k = 0; k < b[l].fi; k++) {
                            if (b[l].first[k] != '@') {
                                for (int n = 0; n < c; n++) {
                                    if (t[n] == b[l].first[k])
                                        strcpy(b[i].par[n], b[i].t[j]);
                                }
```

```
                    } else {
                        for (int m = 0; m < b[i].fo; m++) {
                            for (int n = 0; n < c; n++) {
                                if (t[n] == b[i].follow[m])
                                    strcpy(b[i].par[n], "@");
                            }
                        }
                    }
                }
            }
        }
    }
}
/* -------- PARSER -------- */
void manipulate_string() {
    while (1) {
        int top = 0, i = 0;
        printf("\nEnter the string: ");
        scanf("%s", string);

        strcat(string, "$");
        stack[top++] = '$';
        stack[top++] = b[0].node;
        printf("\nSTACK\tINPUT\n");
        while (top > 0) {
            for (int k = 0; k < top; k++)
                printf("%c", stack[k]);
            printf("\t%s\n", &string[i]);
            char top_sym = stack[top - 1];
            char curr = string[i];
            if (top_sym == curr) {
                top--;
                i++;
            }
            else if (isupper(top_sym)) {
                int idx = -1, col = -1;
                for (int j = 0; j < count; j++)
                    if (b[j].node == top_sym)
                        idx = j;
                for (int k = 0; k < c; k++)
                    if (t[k] == curr)
                        col = k;
                if (idx != -1 && col != -1 && b[idx].par[col][0] != '\0') {
                    top--;
```

```
                if (strcmp(b[idx].par[col], "@") != 0) {
                    for (int n = strlen(b[idx].par[col]) - 1; n >= 0; n--)
                        stack[top++] = b[idx].par[col][n];
                }
            } else {
                printf("\n**String not accepted**\n");
                break;
            }

        } else {
            printf("\n**String not accepted**\n");
            break;
        }
        if (stack[top - 1] == '$' && string[i] == '$') {
            printf("\n***String accepted***\n");
            break;
        }
    }
    printf("\nDo you want to enter another string (Y/N): ");
    scanf(" %c", &e);
    if (e != 'Y' && e != 'y')
        break;
  }
}
/* -------- MAIN -------- */
int main() {
  printf("Enter the productions:\n");
  for (count = 0;; count++) {
    scanf("%s", a[count]);
    if (strcmp(a[count], "quit") == 0)
        break;
    b[count].node = a[count][0];
    b[count].n = 0;
    b[count].fi = 0;
    b[count].fo = 0;
    b[count].k = 0;
    b[count].g = 0;
    for (int i = 3, j = 0; i < strlen(a[count]); i++) {
        if (a[count][i] == '/') {
            b[count].t[b[count].n][j] = '\0';
            b[count].n++;
            j = 0;
        } else {
            if (!isupper(a[count][i]) && a[count][i] != '@') {
                add_unique(t, &c, a[count][i]);
            }
```

```
            b[count].t[b[count].n][j++] = a[count][i];
        }
    }
    b[count].t[b[count].n][strlen(b[count].t[b[count].n])] = '\0';
}
add_unique(t, &c, '$');
for (int i = 0; i < count; i++)
    if (b[i].k == 0)
        first(i);
for (int i = 0; i < count; i++)
    if (b[i].g == 0)
        follow(i);
get_parse_table();
/* -------- PRINT TABLE -------- */
printf("\nParsing Table:\nNT\t");
for (int i = 0; i < c; i++)
    printf("%c\t", t[i]);
printf("\n---------------------------------\n");
for (int i = 0; i < count; i++) {
    printf("%c\t", b[i].node);
    for (int j = 0; j < c; j++) {
        if (b[i].par[j][0])
            printf("%c->%s\t", b[i].node, b[i].par[j]);
        else
            printf("-\t");
    }
    printf("\n");
}
manipulate_string();
return 0;
}
```

OUTPUT:

```
C:\Windows\System32\cmd.e

D:\SKCET\PCD>gcc predictive.c

D:\SKCET\PCD>a.exe
Enter the productions:
E->TA
A->+TA/@
T->FB
B->*FB/@
F->(E)/i
quit

Parsing Table:
NT      +       *       (       )       i       $
---------------------------------
E       -       -       E->TA   -       E->TA   -
A       A->+TA  -       -       A->@    -       A->@
T       -       -       T->FB   -       T->FB   -
B       B->@    B->*FB  -       B->@    -       B->@
F       -       -       F->(E)  -       F->i    -

Enter the string: i+i*i

STACK   INPUT
$E      i+i*i$
$AT     i+i*i$
$ABF    i+i*i$
$ABi    i+i*i$
$AB     +i*i$
$A      +i*i$
$AT+    +i*i$
$AT     i*i$
$ABF    i*i$
$ABi    i*i$
$AB     *i$
$ABF*   *i$
$ABF    i$
$ABi    i$
$AB     $
$A      $

***String accepted***
```

RESULT:

Thus, the LL(1) parser is successfully implemented in C to compute FIRST and FOLLOW sets, construct the parsing table, and validate the input string.

Ex: No:6	Implementation of Shift Reduced Parser
Date:	

AIM:

To implement Shift Reduced Parser using C programming.

ALGORITHM:

Step 1: Start the program.
Step 2: Define a stack structure and initialize the top of the stack to 0.
Step 3: Read the number of productions n.
Step 4: For each production, read the left-hand side (LHS) and right-hand side (RHS).
Step 5: Read the input string to be parsed.
Step 6: Append $ to the input string.
Step 7: Push $ onto the stack.
Step 8: Display initial stack and input string.
Step 9: Repeat until the stack contains start symbol and $ and input is fully processed:
Step 10: Try to reduce the stack contents by matching with RHS of productions.
Step 11: If a match is found, replace RHS with corresponding LHS (reduce operation).
Step 12: Display stack, remaining input, and the reduce action performed.
Step 13: If no reduction is possible, shift the next input symbol onto the stack.
Step 14: Display stack, remaining input, and the shift action performed.
Step 15: Continue shifting and reducing until acceptance condition is reached.
Step 16: If stack contains $ and start symbol and input is $, accept the string.
Step 17: Display "accept" as output.
Step 18: Stop the program.

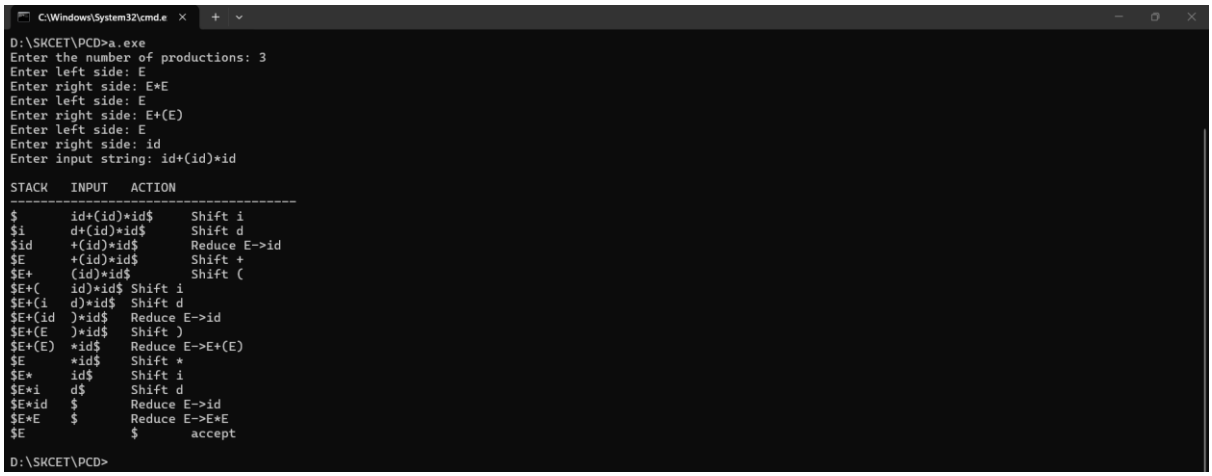
PROGRAM:

```
#include<stdio.h>
#include<string.h>
struct stack {
   char s[20];
   int top;
};
struct stack st;
/* -------- stack functions -------- */
int isempty() {
   return (st.top == 0);
}
void push(char p) {
   st.s[st.top++] = p;
}
char pop() {
   if (!isempty())
      return st.s[--st.top];
   return '\0';
}
void disp() {
   for (int i = 0; i < st.top; i++)
```

```
        printf("%c", st.s[i]);
}
/* -------- reduce -------- */
int reduce(int *j, char rp[10][10], int n, char lp[10]) {
    int i, k;
    char u[20];
    for (i = 0; i < st.top; i++) {
        int len = st.top - i;
        strncpy(u, &st.s[i], len);
        u[len] = '\0';
        for (k = 0; k < n; k++) {
            if (strcmp(rp[k], u) == 0) {
                st.top = i;
                push(lp[k]);
                return k;
            }
        }
    }
    return 99;
}
/* -------- shift -------- */
int shift(char ip[], int *j) {
    push(ip[*j]);
    (*j)++;
    return 1;
}
/* -------- main -------- */
void main() {
    int n, i, j = 0, k, h;
    char lp[10];
    char ip[20];
    char rp[10][10];
    st.top = 0;
    printf("Enter the number of productions: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter left side: ");
        scanf(" %c", &lp[i]);
        printf("Enter right side: ");
        scanf("%s", rp[i]);
    }
    printf("Enter input string: ");
    scanf("%s", ip);
    strcat(ip, "$");
    push('$');
    printf("\nSTACK\tINPUT\tACTION\n");
```

```
    printf("--------------------------------------\n");
    while (1) {
        disp();
        printf("\t%s\t", &ip[j]);
        if ((h = reduce(&j, rp, n, lp)) != 99) {
            printf("Reduce %c->%s\n", lp[h], rp[h]);
        }
        else if (ip[j] != '\0') {
            printf("Shift %c\n", ip[j]);
            shift(ip, &j);
        }
        if (st.top == 2 && st.s[0] == '$' && st.s[1] == lp[0] && ip[j] == '$') {
            disp();
            printf("\t\t%s\taccept\n", &ip[j]);
            break;
        }
    }
}
```

OUTPUT:



```
C:\Windows\System32\cmd.e
D:\SKCET\PCD>a.exe
Enter the number of productions: 3
Enter left side: E
Enter right side: E*E
Enter left side: E
Enter right side: E+(E)
Enter left side: E
Enter right side: id
Enter input string: id+(id)*id

STACK   INPUT   ACTION
--------------------------------------
$       id+(id)*id$     Shift i
$i      d+(id)*id$      Shift d
$id     +(id)*id$       Reduce E->id
$E      +(id)*id$       Shift +
$E+     (id)*id$        Shift (
$E+(    id)*id$ Shift i
$E+(i   d)*id$  Shift d
$E+(id  )*id$   Reduce E->id
$E+(E   )*id$   Shift )
$E+(E)  *id$    Reduce E->E+(E)
$E      *id$    Shift *
$E*     id$     Shift i
$E*i    d$      Shift d
$E*id   $       Reduce E->id
$E*E    $       Reduce E->E*E
$E              $       accept

D:\SKCET\PCD>
```

RESULT:

Thus, the shift-reduce parser is successfully implemented using c programming to check whether the input string is accepted or rejected.

Ex: No:7	Implementation of SLR Parser
Date:	

AIM:

To implement the SLR parser using C Programming.

ALGORITHM:

Step 1: Start the program.
Step 2: Read non-terminals, terminals, and construct the parsing table.
Step 3: Display the parsing table.
Step 4: Read productions and store LHS and RHS.
Step 5: Read the input string and append $.
Step 6: Initialize the stack with $ and starting state 0.
Step 7: Repeat parsing by checking action from table using stack top and input symbol.
Step 8: Perform Shift or Reduce operation accordingly and update stack.
Step 9: Display stack and remaining input after each step.
Step 10: If stack and input reach $, accept the string; otherwise, reject.
Step 11: Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
struct table {
   char op;   // S, R, A, or -
   int value;
   char set;
} ta[50][50];
int main() {
   char nt[20], t[20], ip[50], s[100];
   char prod[20][20], rhs[20][20], lhs[20][20];
   int non, not, ni, np;
   int i, j, l, ch, tos, temp1;
   printf("Enter number of non-terminals: ");
   scanf("%d", &non);
   for (i = 0; i < non; i++) {
      printf("Enter non-terminal: ");
      scanf(" %c", &nt[i]);
   }
   printf("Enter number of terminals: ");
   scanf("%d", &not);

   for (i = 0; i < not; i++) {
      printf("Enter terminal: ");
      scanf(" %c", &t[i]);
      nt[non + i] = t[i];
   }
   nt[non + not] = '$';
```

```
nt[non + not + 1] = '\0';
printf("Enter number of states: ");
scanf("%d", &ni);
// Input parsing table
for (i = 0; i < ni; i++) {
    printf("Enter number of entries in state %d: ", i);
    int entries;
    scanf("%d", &entries);
    for (l = 0; l < entries; l++) {
        char symbol;
        printf("Enter symbol: ");
        scanf(" %c", &symbol);
        int col = -1;
        for (j = 0; nt[j] != '\0'; j++) {
            if (nt[j] == symbol) {
                col = j;
                break;
            }
        }
        printf("Enter operation (S/R/A): ");
        scanf(" %c", &ta[i][col].op);
        printf("Enter step: ");
        scanf("%d", &ta[i][col].value);
        ta[i][col].set = 't';
    }
}
// Print table
printf("\nParsing Table:\n");
for (i = 0; nt[i] != '\0'; i++)
    printf("%c\t", nt[i]);
printf("\n");

for (i = 0; i < ni; i++) {
    for (j = 0; nt[j] != '\0'; j++) {
        if (ta[i][j].set == 't')
            printf("%c%d\t", ta[i][j].op, ta[i][j].value);
        else
            printf("\t");
    }
    printf("\n");
}
// Productions
printf("Enter number of productions: ");
scanf("%d", &np);
for (i = 0; i < np; i++) {
    printf("Enter production (e.g. E->E+T): ");
```

```
    scanf("%s", prod[i]);
    lhs[i][0] = prod[i][0];
    lhs[i][1] = '\0';
    strcpy(rhs[i], prod[i] + 3);
}
int ch_flag = 1;
while (ch_flag == 1) {
    printf("\nEnter input string: ");
    scanf("%s", ip);
    int len = strlen(ip);
    ip[len] = '$';
    ip[len + 1] = '\0';
    strcpy(s, "$0");
    tos = 1;
    i = 0;
    printf("\nStack\tInput\tAction\n");
    while (1) {
        printf("%s\t%s\t", s, ip + i);
        int state = s[tos] - '0';

        // Find column
        int col = -1;
        for (j = 0; nt[j] != '\0'; j++) {
            if (nt[j] == ip[i]) {
                col = j;
                break;
            }
        }
        if (col == -1) {
            printf("ERROR\n");
            break;
        }
        char action = ta[state][col].op;
        temp1 = ta[state][col].value;
        if (action == 'S') {
            printf("SHIFT %d\n", temp1);
            s[++tos] = ip[i++];
            s[++tos] = temp1 + '0';
            s[tos + 1] = '\0';
        }
        else if (action == 'R') {
            int prodIndex = temp1 - 1;
            int popCount = 2 * strlen(rhs[prodIndex]);
            printf("REDUCE %s->%s\n", lhs[prodIndex], rhs[prodIndex]);
            // Pop RHS
            while (popCount--)
```

```
            s[tos--] = '\0';
        // Previous state
        int prevState = s[tos] - '0';
        // Push LHS
        s[++tos] = lhs[prodIndex][0];
        // Find GOTO column
        int gotoCol = -1;
        for (j = 0; nt[j] != '\0'; j++) {
            if (nt[j] == lhs[prodIndex][0]) {
                gotoCol = j;
                break;
            }
        }
        int gotoState = ta[prevState][gotoCol].value;
        s[++tos] = gotoState + '0';
        s[tos + 1] = '\0';
    }
    else if (action == 'A') {
        printf("ACCEPTED\n");
        break;
    }
    else {
        printf("ERROR\n");
        break;
    }
  }
  printf("Do you want to enter another string? (1/0): ");
  scanf("%d", &ch_flag);
 }
 return 0;
}
```

OUTPUT:

```
C:\Windows\System32\cmd.e

D:\SKCET\PCD>gcc slr.c

D:\SKCET\PCD>a.exe
Enter number of non-terminals: 2
Enter non-terminal: S
Enter non-terminal: A
Enter number of terminals: 2
Enter terminal: a
Enter terminal: b
Enter number of states: 4
Enter number of entries in state 0: 2
Enter symbol: b
Enter operation (S/R/A): S
Enter step: 2
Enter symbol: A
Enter operation (S/R/A): -
Enter step: 1
Enter number of entries in state 1: 1
Enter symbol: a
Enter operation (S/R/A): S
Enter step: 3
Enter number of entries in state 2: 1
Enter symbol: a
Enter operation (S/R/A): R
Enter step: 2
Enter number of entries in state 3: 1
Enter symbol: $
Enter operation (S/R/A): A
Enter step: 0

Parsing Table:
S       A       a       b       $
        -1              S2
                S3
                R2
                                A0
Enter number of productions: 2
Enter production (e.g. E->E+T): S->Aa
Enter production (e.g. E->E+T): A->b

Enter input string: ba

Parsing Table:
S       A       a       b       $
        -1              S2
                S3
                R2
                                A0
Enter number of productions: 2
Enter production (e.g. E->E+T): S->Aa
Enter production (e.g. E->E+T): A->b

Enter input string: ba

Stack   Input   Action
$0      ba$     SHIFT 2
$0b2    a$      REDUCE A->b
$0A1    a$      SHIFT 3
$0A1a3  $       ACCEPTED
Do you want to enter another string? (1/0):
```

RESULT:

Thus, the SLR parsing table is successfully constructed and the given input string is analyzed to determine whether it is accepted or rejected based on the grammar.

Ex: No:8	Implementation of front end of a compiler that generates the three address code
Date:	

AIM:

To implement the front end of a compiler that generates the three address code for a simple language.

ALGORITHM:

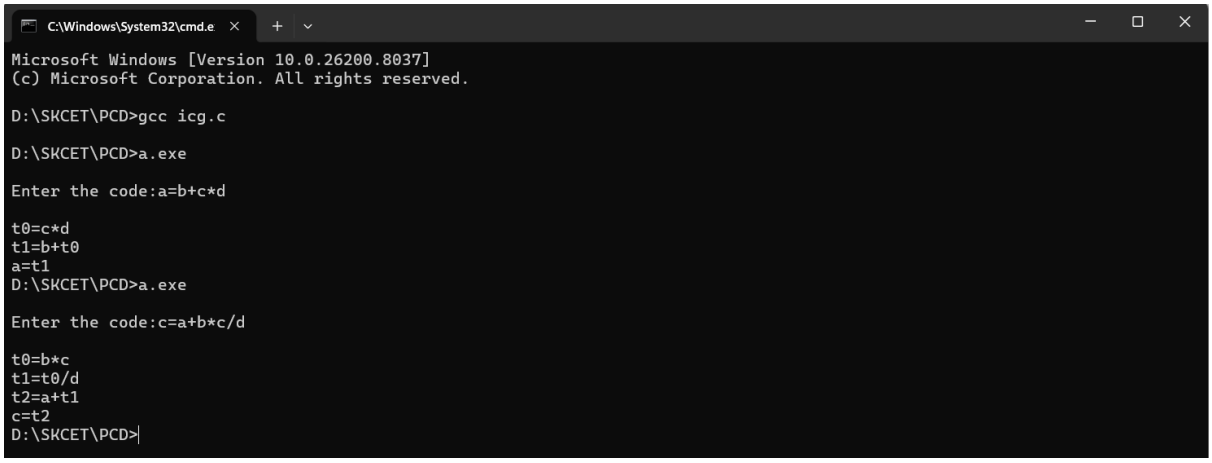
Step 1: Start the program.
Step 2: Read the input arithmetic expression.
Step 3: Copy the input expression into another string for processing.
Step 4: Initialize temporary variable counter t0, t1, ...
Step 5: Scan the expression for * and / operators.
Step 6: For each * or /, identify its operands and form a temporary expression.
Step 7: Generate intermediate code using a temporary variable and replace the sub-expression in the string.
Step 8: Repeat the process until all * and / operations are processed.
Step 9: Scan the updated expression for + and - operators.
Step 10: For each + or -, identify operands, generate temporary variable, and replace the sub-expression.
Step 11: Continue until the entire expression is reduced to a single assignment.
Step 12: Display the generated intermediate code and stop the program.

PROGRAM:

```
#include<stdio.h>
#include<ctype.h>
#include<string.h>
int ag=0,z=1;
void main(){
char
a[50],id[50],b[50],op[50],mov[]="MOVF",mul[]="MULF",div[]="DIVF",add[]="ADDF",sub[]="
SUBF",ti=0;
int i=0,j=0,k=0,len=0,s=0,e=0,r=1,count;
FILE *fp;
fp=fopen("out.txt","w");
printf("\nEnter the code:");
scanf("%s",a);
strcpy(b,a);
len=strlen(a);
for ( i=0;i<strlen(b);i++ ){
if ( b[i] == '*' || b[i] == '/' ){
for ( j=i-1;b[j]!='-'&&b[j]!='+'&&b[j]!='*'&&b[j]!='/'&&b[j]!='=';j--);
k=j+1;
count=0;
printf("\nt%d=",ti++);
for ( j=j+1;count<2&&b[j]!='\0';j++ ){
```

```
if ( b[j+1] == '+' || b[j+1] == '-' || b[j+1] == '*' || b[j+1] =='/' )
count++;
printf("%c",b[j]);}
b[k++]='t';
b[k++]=ti-1+48;
for ( j=j,k=k;k<strlen(b);k++,j++ )
b[k]=b[j];
i=0;}}
for ( i=0;i<strlen(b);i++ ){
if ( b[i] == '+' || b[i] == '-' ){
for ( j=i-1;b[j]!='-'&&b[j]!='+'&&b[j]!='=';j--);
k=j+1;
count=0;
printf("\nt%d=",ti++);
for ( j=j+1;count<2&&b[j]!='\0';j++ ){
if ( b[j+1] == '+' || b[j+1] == '-' )
count++;
printf("%c",b[j]);}
b[k++]='t';
b[k++]=ti-1+48;
for ( j=j,k=k;k<strlen(b);k++,j++ )
b[k]=b[j];}}
printf("\n%s",b);}
```

OUTPUT:



RESULT:

Thus, intermediate code generation is successfully implemented using temporary variables.

Ex: No:9	Implementation of the back end of the compiler
Date:	

AIM:

To implement the backend of the complier which generate the assembly code.

ALGORITHM:

Step 1: Start the program.
Step 2: Read the input arithmetic expression.
Step 3: Open the output file to store generated code.
Step 4: Initialize register counter for intermediate results.
Step 5: Scan the expression to detect the assignment operator =.
Step 6: Identify identifiers and constants in the expression.
Step 7: Generate MOV instructions to load operands into registers.
Step 8: Scan the expression from right to left for operators (+, -, *, /).
Step 9: For each operator, generate corresponding instructions (ADDF, SUBF, MULF, DIVF).
Step 10: Perform operations using registers for intermediate results.
Step 11: Store the final result into the destination variable using MOVF.
Step 12: Display the generated code and stop the program.

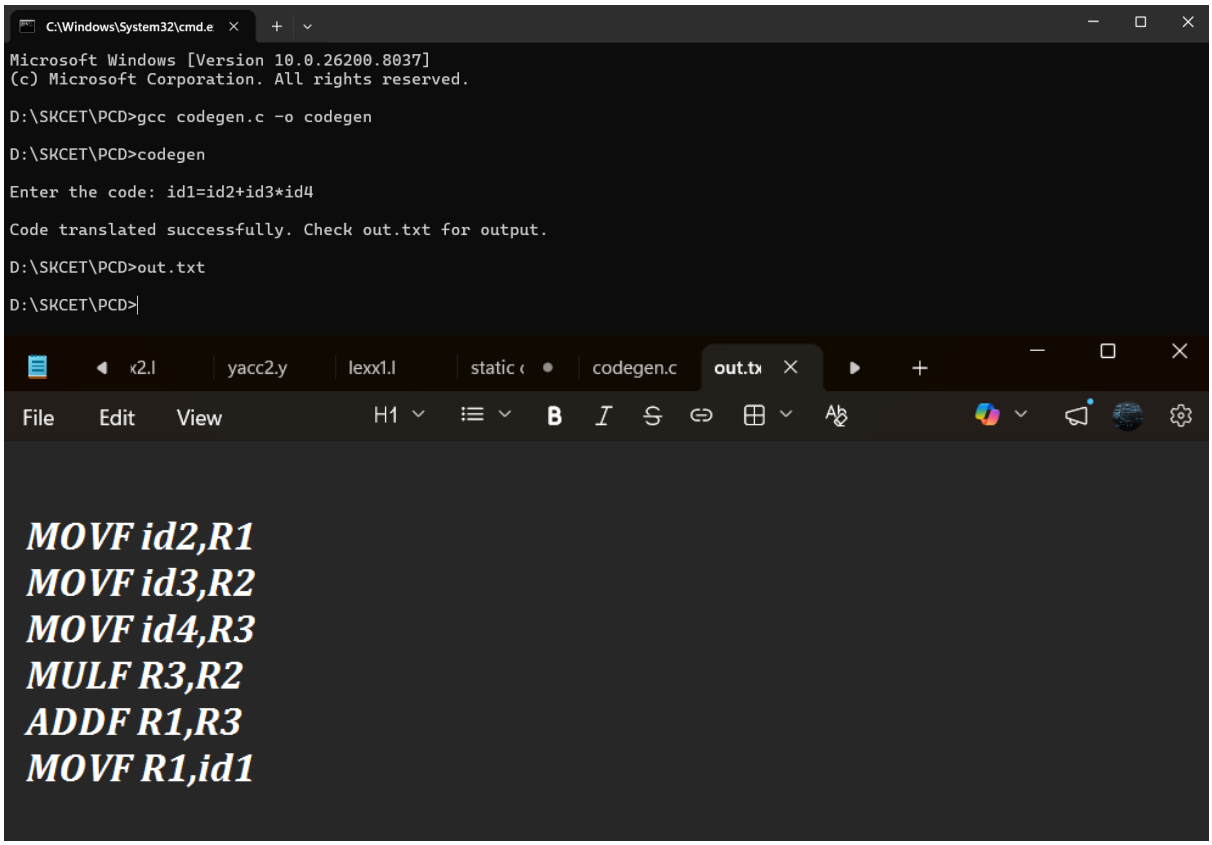
PROGRAM:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
int ag = 0, z = 1;
int main() {
   char a[50], id[50];
   char mov[] = "MOVF", mul[] = "MULF", div[] = "DIVF", add[] = "ADDF", sub[] = "SUBF";
   int i = 0, j = 0, len = 0, s = 0, e = 0, r = 1;
   FILE *fp;
   fp = fopen("out.txt", "w");
   if (fp == NULL) {
      printf("Error opening file!\n");
      return 1;
   }
   printf("\nEnter the code: ");
   fgets(a, sizeof(a), stdin);   // safer than gets()
   len = strlen(a);
   // Handle MOV instructions
   for (i = 0; i < len; i++) {
      if (a[i] == '=') {
         for (j = i; j < len; j++) {
            if (a[j] == 'i') {
               fprintf(fp, "\n%s ", mov);
               fprintf(fp, "%c%c%c,R%d", a[j], a[j + 1], a[j + 2], r++);
            }
```

```
        }
    } else if (isdigit(a[i]) && isdigit(a[i + 1])) {
        fprintf(fp, "\n%s #%c%c,R%d", mov, a[i], a[i + 1], r++);
    }
}
// Handle arithmetic operations
for (i = len - 1; i >= 0; i--) {
    if (a[i] == '+') {
        fprintf(fp, "\n%s ", add);
        e = a[i - 1];
        e--;
        s = e;
        if (a[i + 1] == 'i')
            fprintf(fp, "R%c,R%d", e, r - 1);
    } else if (a[i] == '-') {
        fprintf(fp, "\n%s ", sub);
        e = a[i - 1];
        e--;
        s = e;
        if (a[i + 1] == 'i')
            fprintf(fp, "R%c,R%c", (a[i + 3] - 1), s);
        else
            fprintf(fp, "R%c,R%d", e, r - 1);
    } else if (a[i] == '*') {
        fprintf(fp, "\n%s ", mul);
        e = a[i - 1];
        e--;
        s = e;
        if (a[i + 1] == 'i')
            fprintf(fp, "R%c,R%c", (a[i + 3] - 1), s);
        else
            fprintf(fp, "R%c,R%d", e, r - 1);
    } else if (a[i] == '/') {
        fprintf(fp, "\n%s ", div);
        e = a[i - 1];
        e--;
        s = e;
        if (a[i + 1] == 'i')
            fprintf(fp, "R%c,R%c", (a[i + 3] - 1), s);
        else
            fprintf(fp, "R%c,R%d", e, r - 1);
    }
}
fprintf(fp, "\n%s R1,id1", mov);
fclose(fp);
printf("\nCode translated successfully. Check out.txt for output.\n");
```

```
    return 0;
}
```

OUTPUT:



RESULT:

Thus, target code generation is successfully implemented using registers for the given expression.

Ex: No:10	Implementation of Code optimization
Date:	

AIM:

To implement the code optimization of compiler using C programming.

ALGORITHM:

Step 1: Start the program.
Step 2: Read the number of expressions and store LHS and RHS of each statement.
Step 3: Display the given intermediate code.
Step 4: For each statement, check whether its variable is used in other expressions.
Step 5: Store only the useful statements to perform dead code elimination.
Step 6: Display the code after removing dead code.
Step 7: Compare RHS of expressions to find common sub-expressions.
Step 8: If common expressions are found, replace them with a single variable (common subexpression elimination).
Step 9: Update remaining expressions with the replaced variables.
Step 10: Remove duplicate or redundant expressions.
Step 11: Display the optimized intermediate code.
Step 12: Stop the program.

PROGRAM:

```
#include <stdio.h>
#include <string.h>
struct op {
   char l;
   char r[20];
} op[10], pr[10];
int main() {
   int i, j, n, z = 0;
   char *p, temp;
   printf("\nEnter the number of values: ");
   scanf("%d", &n);
   for (i = 0; i < n; i++) {
      printf("\nLeft: ");
      scanf(" %c", &op[i].l);
      printf("Right: ");
      scanf("%s", op[i].r);
   }
   printf("\nIntermediate code\n");
   for (i = 0; i < n; i++)
      printf("%c = %s\n", op[i].l, op[i].r);

   // Dead code elimination
   for (i = 0; i < n; i++) {
      temp = op[i].l;
      p = NULL;
```

```
        for (j = 0; j < n; j++) {
            p = strchr(op[j].r, temp);
            if (p) {
                pr[z].l = op[i].l;
                strcpy(pr[z].r, op[i].r);
                z++;
                break;
            }
        }
    }
    printf("\nAfter dead code elimination\n");
    for (i = 0; i < z; i++)
        printf("%c = %s\n", pr[i].l, pr[i].r);
    // Common subexpression elimination (fixed)
    for (i = 0; i < z; i++) {
        for (j = i + 1; j < z; j++) {
            if (strcmp(pr[i].r, pr[j].r) == 0) {
                char tempStr[2];
                tempStr[0] = pr[i].l;
                tempStr[1] = '\0';
                strcpy(pr[j].r, tempStr);
            }
        }
    }
    printf("\nEliminate common expression\n");
    for (i = 0; i < z; i++)
        printf("%c = %s\n", pr[i].l, pr[i].r);
    printf("\nOptimized code\n");
    for (i = 0; i < z; i++)
        if (pr[i].l != '\0')
            printf("%c = %s\n", pr[i].l, pr[i].r);
    return 0;
}
```

OUTPUT:

```
C:\Windows\System32\cmd.e

D:\SKCET\PCD>gcc codeop.c

D:\SKCET\PCD>a.exe

Enter the number of values: 3

Left: x
Right: a+b

Left: y
Right: a+b

Left: z
Right: x+y

Intermediate code
x = a+b
y = a+b
z = x+y

After dead code elimination
x = a+b
y = a+b

Eliminate common expression
x = a+b
y = x

Optimized code
x = a+b
y = x

D:\SKCET\PCD>
```

RESULT:

Thus, the given intermediate code is optimized by eliminating dead code and common sub-expressions, resulting in a more efficient final code.